8.1.3 Use cases

Distributed tracing can be useful not just in distributed systems where services call others over the network but also for monolithic apps. In general, you can benefit from it in the following cases:

- *Monitoring microservices-based applications*—These allow you to acquire an end-to-end overview of the actual architecture of your app, including call dependencies. Rather than relying on static architecture diagrams of how the app is supposed to work, you get a visual representation of how the app actually works. You can determine error states (what services error out) and performance issues (latency graphs of service invocations) of services, usually with a single glance at a service map.
- *Optimization*—This is used to determine where along the request path most of the time is spent. You can pinpoint the slowest service and start the latency optimization there, promising the most yield in terms of investment.
- *Release management*—When rolling out a new version of a service, you can assess the impact on the overall request paths the service is part of, comparing before and after the new version.
- *Troubleshooting*—In the case of an incident, as we discussed in chapter 7, you can use the distributed tracing solution to figure out which services contribute to the issue and can roll them back to a known good state to stop the bleeding. We will see this common use case in action in the next subsection of this chapter.

OK, that was more than enough theory! Let's get our hands dirty with distributed tracing, focusing on the troubleshooting use case.

8.2 Using distributed tracing in a microservices app

In this section, you will use distributed tracing in the context of a simple example microservices app. The goal is to understand how the services work together and, once we start simulating errors, use distributed tracing to identify which services cause issues and what kind of errors you encounter.

8.2.1 Example app overview

Our example app (to be set up later) consists of four services, as depicted in figure 8.4. We have a frontend service as the entry point, which calls two services: service `A` and service `B`; in turn, service `B` calls two other services (`B0` and `B1`). Some of the services also call out to static assets on the web to simulate calls to external third-party APIs. Note that our example app doesn't do anything useful; all of the services use the ho11y (http://mng.bz/rWxg) synthetic signal generator to generate traces, and we can configure it to simulate certain failure modes.

Our example app stack contains the five services making up the microservices app as well as the OpenTelemetry collector to ingest the spans generated from the four